PRIMAD: Building Digital Twins for Software Development Lifecycle within an Evolutionary Architecture

Miguel Ángel Guinea-Cabrera
Software Engineering Department.
Research Centre for Information and Communication Technologies (CITIC-UGR)
University of Granada
Granada, Spain
https://orcid.org/0009-0009-8479-3610

Juan Antonio Holgado-Terriza
Software Engineering Department.
Research Centre for Information and Communication Technologies (CITIC-UGR)
University of Granada
Granada, Spain
https://orcid.org/0000-0002-8031-1276

Pablo A. Pico-Valencia
Software Engineering Department.
Research Centre for Information and Communication Technologies (CITIC-UGR)
University of Granada
Granada, Spain
https://orcid.org/0000-0003-3518-3313

2025 IEEE Smart World Congress (SWC) | 979-8-3315-7598-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/SWC65939.2025.00261

***Abstract*— Current Digital Twin (DT) adoption in the Software Development Lifecycle (SDLC) faces significant hurdles, including a lack of standardized methods, limited research, and isolated use cases. Optimizing value streams is crucial for modern software organizations, which are increasingly characterized by diverse and evolving roles. Implementing a digital twin in this context can significantly enhance the ability to identify opportunities for improvement and explore potential change scenarios. To overcome these adoption barriers, we propose PRIMAD, a progressive methodology for building an SDLC DT that delivers incremental benefits by leveraging existing empirical data from issue-tracking systems, such as JIRA. This approach resolves the "cold start" problem, enabling simulations to emerge directly from real-world behavior. Our methodology integrates a Python-based simulation core with an Akka actor system for modeling issues and resources, enriched by supervised machine learning for context-aware predictions. Preliminary results validating PRIMAD components independently (using pm4py, Akka, and LightGBM) show promising outcomes. This paper details our methodology, presents preliminary component-wise results, and outlines future work.**

Keywords: digital twin, software agents, actor model, software engineering, machine learning.

INTRODUCTION

A preliminary systematic literature review revealed a scarcity of Digital Twins (DTs) in the software industry, resulting in a lack of proven methods for building them and limited technological options to achieve this goal [1]. Previous research also identified a few isolated use cases where DTs were implemented within the Software Development Lifecycle (SDLC). From their inception, DTs have emerged as a powerful outcome of digital transformation [2]. In [2], the authors outline a series of steps—also followed in our research—regarding the applicability of DTs to the SDLC, including architectural components and a method for constructing them. This paper presents preliminary component-wise results and outlines future directions.

The SDLC involves a variety of virtual assets and complex human interactions, which must be accurately reflected in the DTs. These DTs should enable the observation of the System under Study (SuS), specifically its emergent behavior as issues traverse the lifecycle. Furthermore, they should also support the evaluation of changes to various lifecycle components to provide relevant insights for the real system. The level of integration between the DT and the real entities determines their typology [3]. However, authors in [4] identify the high cost of DT implementation as a significant barrier to its adoption. Consequently, the selected approach must deliver incremental benefits and support gradual implementation to justify its investment. Although there is research discussing the use of DT at the organizational level, a clear implementation roadmap is still lacking [5].

Non-functional requirements, such as scalability and availability, drive the range of approaches available for implementation, particularly in leveraging the simulation capabilities of DTs. The selected paradigms must effectively model SuS, capturing relationships and behaviors among virtual entities and human participants. The current state of Artificial Intelligence (AI) heavily relies on distributed intelligent systems, which integrate cognitive artifacts capable of fluid human interaction and meaningful contributions to decision-making processes [6]. This positions both agents and actors as strong candidates for the development of systems that enable decentralized AI. In this context, Minerva et al. [7] highlight the overlapping functionalities between Multi-Agent Systems (MAS) and DTs.

Capitalizing on the inherent suitability of the agent and actor models for DTs in the SDLC, this study adopts Agent-Based Modeling (ABM). This bottom-up approach to visualizing system behavior, proven in social sciences [8], promotes generative simulation design grounded in empirical discovery [9]. To this end, our research uses a publicly available JIRA dataset [10] to identify the implicit SDLC within issue tracking flows and assess the applicability of DTs in replicating this flow. Our goal is to demonstrate how DTs can provide tangible value to organizations through forecasts and insights that reduce cycle time and accelerate the value stream.

As a preliminary innovation of this work, we introduce the methodology PRIMAD (Process Mining, Refinement, Intelligence, Multi-agent Simulation Architecture and Dynamic Extension) as a progressive approach for building a DT for the SDLC that overcomes the cold start problem by leveraging existing empirical data from issue-tracking systems like JIRA, offering a practical pathway for organizations to realize the benefits of DTs in their software

2993-396X/25/$31.00 ©2025 IEEE
DOI 10.1109/SWC65939.2025.00261

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:01:58 UTC from IEEE Xplore. Restrictions apply.

development processes. Another study explores the combination of process mining and DTs, as described in [11].

Minerva et al. [7] highlight the functional similarities between MAS and DTs. They posit that both actors and agents support parallel and distributed computation in AI, employing message-passing mechanisms for communication [12]. Nonetheless, actors are computational entities that operate by asynchronously sending and receiving messages. They are reactive by nature and only execute tasks in response to the messages they receive. In contrast, agents are autonomous entities embedded within an environment and are capable of reactive, proactive, and social behavior [13]. Their design emphasizes human-like capabilities, including reasoning and planning. Furthermore, agents often incorporate learning algorithms to improve communication and coordination, even in uncertain environments [14].

Although both actors and agents contribute to AI system development, the actor model is considered "weaker" in terms of expressiveness compared to the agent model [12]. While the actor model ensures autonomy and reactivity, modeling proactive behavior is not straightforward when using pure actors. Similarly, actors inherently lack reasoning capabilities, although such logic can be embedded in their internal processes [12]. A hybrid architecture tested in [15] combined a deliberative actor model with frameworks such as Jason, a BDI (Belief-Desire-Intention) MAS platform, utilizing the actor model as its infrastructure. The actor model demonstrated superior performance compared to MAS, primarily due to its focus on parallelism and ease of integration with distributed systems. In this regard, the Akka framework provides valuable features such as high availability, persistence, and scalability [16]. Notably, its architecture enables multiple actors to run within a single thread—an essential advantage over MAS platforms, such as the Java Agent Development Framework (JADE) [17].

This paper is organized into five sections. Section 2 presents the key concepts of the DT framework for modeling the SDLC. Section 3 outlines our proposed approach, called PRIMAD. Section 4 presents the quantitative and qualitative results derived from analyzing a public JIRA dataset in the context of the SDLC, along with insights from a real-world case application of the machine learning classification modeling steps described in PRIMAD. Finally, Section 5 presents the conclusions and outlines future work.

ARCHITECTURE

This study proposes a DT framework for modeling and simulating the SDLC, inspired by the ISO 23247-2:2021 reference architecture [1]. The approach integrates a Python-based simulation core with an Akka actor system for modeling issues and resource dynamics, synchronizing with JIRA as the system of record. The modular architecture comprises functional entities, including simulation, analytics, synchronization, and system interoperability.

This design enables real-time observation, predictive experimentation, and dynamic feedback to optimize workflows and decision-making in software engineering environments. All timing is transformed and stored in a specific time zone, considering two dimensions: the real dimension in the SuS and the dimension applied to the simulation to facilitate time travel. The main elements of entities are represented in Table I.

TABLE I. ARCHITECTURAL MAPPING BASED ON ISO 23247-2:2021.

ID	Modular architecture vs ISO 23247-2:2021	
	ISO Entity	***Description***
UE	User Entity	UI dashboards for controlling and visualizing the twin
CE.OM	Core Entity: Operation & Management Sub-Entity	Configuration of resources, rules, and roles; presentation and version control of the DT
CE.AS	Core Entity: Application & Service Sub-Entity	Core simulation engine, analytics, and prediction services handling
CE.RAI	Core Entity: Resource Access & Interchange Sub-Entity	APIs and connectors for SuS, handle security, and data transformation
DC.C	Data Collection & Device Control: Collection Sub-Entity	Collects and pre-processes real-time data from SDLC tools
DC.DC	Data Collection & Device Control: Device Control Sub-Entity	Control actions applicability on SuS based on simulation output
CS	Cross-System Entity	Ensures security, data quality, and translation across heterogeneous environments
OME	Observable Manufacturing Elements	Real-world elements like issues and resources are represented in the twin

As exposed in [4], the high cost of implementation and the need to deliver gradual results are significant concerns. Consequently, an evolutionary architecture approach, suggested in [18] and [19], is identified as a key trend [20].

Table II illustrates the implementation of each proposed framework element and the outcomes generated at each phase. This evolutionary strategy enables rapid initial outcomes and a gradual progression towards a fully scalable scenario with high availability and massive scale in projects, issues, and resources.

TABLE II. ARCHITECTURAL EVOLUTIONARY PATH.

Phase	Evolutionary architecture roadmap	
	Description	***Goal achieved***
Python modules + 1 Akka microservice	Start everything in a single repository. Python orchestrates the logic (Core Entity) and calls the Akka microservice (Simulation Service)	Fast prototyping and idea validation
Internal modularization	Organizing code into folders representing the ISO entities	Preparing for future separation without significant refactoring
Progressive extraction	Gradually move critical services (like SuS connector, UI dashboard) into independent containers or microservices	Scalability + independent testing
Event-driven architecture	Introduce message queues to decouple service flows fully	Enables parallel project analysis and asynchronous scenarios
Large-scale distributed simulation	The Akka cluster handles thousands of actors; Python orchestrates multiple simulation instances	Massive scalability and distributed simulation capabilities

Figure 1 illustrates the first architectural phase, as described in Tables I and II, which replaces the concept of "Device" in ISO 23247-2:2021 with the idea of "System under

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:01:58 UTC from IEEE Xplore. Restrictions apply.

Study" (SuS). The dashed-line boxes in the figure represent PRIMAD steps, which are designed to build the core elements in the architecture and the DT simulation capabilities. PRIMAD leverages recent history from OME (Table I) to develop or update content for key architectural aspects, thereby avoiding the cold start problem. The UE layer enables manual fine-tuning of parameters and key values for the function on digital twins. This structure allows different DT versions to run in parallel. This is a significant benefit, as the other DT parametrization helps us understand how the SDLC works in various scenarios (with varying allocations of teams, resources, and availability) and its impact on throughput. The main architecture is extended to retain these distinct DT instances. The configuration ensures that only one active DT sends actions back to the OME at any given time.

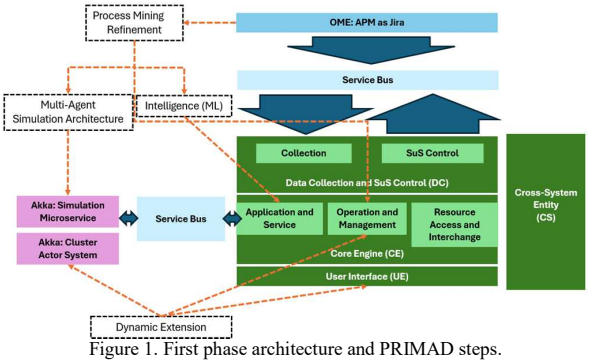

Figure 1. First phase architecture and PRIMAD steps.

PRIMAD Approach

The PRIMAD acronym outlines the steps followed to build a DT for SDLC in a progressive approach: Process Mining, Refinement (Preprocessing), Intelligence (Machine Learning Model), Multi-agent Simulation Architecture (Digital Twin), and Dynamic Extension (Rules, Large Language Models, Live Edits). This methodology addresses the need to resolve the cold start problem on DT, enabling the simulation to emerge directly from empirical data. For SDLC, the JIRA dataset contains many issues from different projects. This is frequently seen in many companies. This approach requires having some data available already in JIRA or other issue-tracking software. The resulting DT will emerge from the existing data as a starting point.

Process Mining

The pm4py library [21], a specialized process mining library, extracts relevant information, including resources, states, frequencies, timings, and sojourn times. Its primary purpose is to address the cold start issue for DTs that lack sufficient data about the SuS. For its usage in pm4py, JIRA data must be converted to event log format, which requires thorough cleaning to address inconsistencies, such as differences in capitalization. The data period should be recent and provide sufficient volume to capture a diverse range of cases for robust process mining. Among available algorithms, the inductive miner is preferred for its robustness and ability to handle data noise. This step yields key information about the frequency of transitions and the resources related to each state, including the sojourn time (the time spent in each state).

Refinement (Preprocessing)

Pm4py can extract highly relevant information to avoid a cold start and facilitate the construction of the CE.OM (Core Entity: Operation & Management Sub-Entity) from Table I. This information, combined with the previously generated event log, can be further processed to uncover hidden process, such as cycle time and activity duration per resource, which are relevant to the process. For the next step, a new file must be created for machine learning training. This file will consolidate the most pertinent issue attributes, including data extracted from the event log, such as current state, next state, and associated resource.

Intelligence (Machine Learning Classification Models)

To enrich the simulation (CE.AS, Core Entity: Application & Service Sub-Entity, in Table I) with intelligent behavior, we integrate supervised machine learning models that predict either the next state or the following resource from current event log data. Instead of relying solely on statistical transition frequencies derived from process mining, a Gradient Boosting classifier (LightGBM) provides probabilistic, context-aware predictions. This model is trained using an enriched event log with structured data: (i) current state, (ii) current resource, (iii) activity name and any available description, (iv) case attributes (priority, issue_type, project), (v) time-related features, (vi) following resource, and (vii) next state.

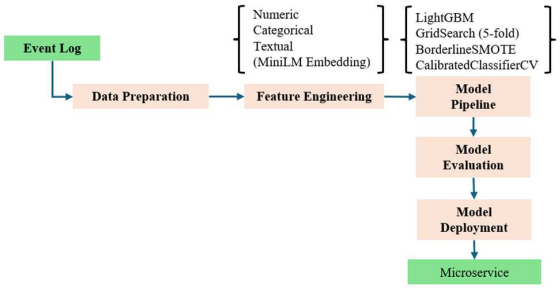

Figure 2. Pipeline overview for the machine learning model.

Figure 2 illustrates the systematic steps involved in training and testing our machine learning model, as follows:

- Data preparation: The dataset is filtered to remove rare classes. The fields (following resource or next state) to predict are encoded using LabelEncoder.
- Feature Engineering: In this step, the attributes are processed to train the model. Numeric features are standardized, categorical features are one-hot encoded, and textual features are embedded using SentenceTransformer (MiniLM model).
- Modeling and Tuning: A LightGBMClassifier [22] is embedded within a Pipeline. GridSearchCV is used for hyperparameter optimization with 5-fold stratified cross-validation. The best hyperparameters are extracted for final processing.
- Class imbalance handling: The features matrix is oversampled using BorderlineSMOTE for borderline cases.
- Calibration: The best LightGBM model is a wrapper with CalibratedClassifierCV [22] to improve probability estimates.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:01:58 UTC from IEEE Xplore. Restrictions apply.

- Evaluation: Metrics from the train-test split are reviewed from the classification report, including accuracy and F1-score.
- Deployment: The final model predicts the top 1 or 3 class labels along with their associated confidence scores. The obtained models are then serialized and stored. Consequently, each model is encapsulated within a microservice that can be consumed by elements within the DT simulation. Each microservice maintains its database independently, tracking usage and results.

This method extracts the probability for every class and the overall prediction confidence. If this confidence is low, purely statistical information can be used. In such a case, the suitable approach involves employing two independent models: one to predict the new state and another for the subsequent resource. This dual-model strategy significantly increases prediction accuracy.

As shown in the indicated steps with SentenceTransformer and the preprocessor, a step in the microservices must be created to prepare the data before model inference. This step is also stored together with the model.

Multi-Agent Simulation Architecture

The simulation requires high availability, as issue-tracking systems like JIRA work based on this requirement in software companies with teams in different time zones. In this case, the actor model implemented with Akka presents significant scalability and high availability benefits. A microservice architecture for this service is ideal for completing CE.AS as outlined in Table I.

For the first DT model iteration, the suggested approach to follow consists of the following main types of actors:

- Issue Twin Actor: These actors represent issues that arise in the SDLC. Implement using Akka Finite State Machines (FSMs), their states, transition frequencies, and occurrence rates are identified via pm4py. They consume microservices for the next step and resource predictions, leveraging available sojourn time data. Akka facilitates seamless integration with external services straightforwardly and conveniently.
- ResourceActor: These actors represent various resources (e.g., developers, testers) interacting with system issues. Their deliberative behavior can be achieved by directly associating rules with each actor, or by modeling rules using BDI frameworks, such as Jason [15], to simulate rational behavior. A rule engine, such as Drools, can assist the ResourceActor in executing the rules and understanding the necessary actions. Some attributes, such as availability, timetable, and timezone, could be included to enhance the simulation.
- SimulationManagerActor: This primary actor orchestrates the simulation by instantiating and supervising all other actors.

Akka actors will register within an Akka cluster, thereby achieving the high availability requirements as described in Figure 3. The information extracted from the simulation will be stored in a graph database, such as OrientDB [24]. This database will also store data on issues and actors, as well as backup information.

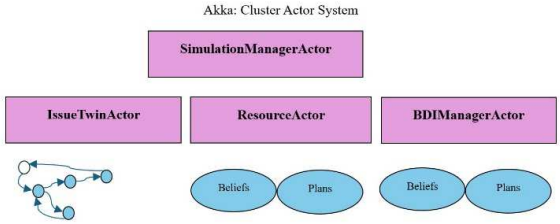

Figure 3. Akka cluster with representation of leading actors.

In the approach to select whether to use a BDI framework like Jason or a rules engine like Drools, there are insights based on the research that can drive the decision:

- BDI frameworks, such as Jason, are not easily integrated into an Akka cluster, which could limit the achievement of a high-availability solution.
- The Rules Engine approach facilitates the hot reload of rule changes into the simulation, providing extra flexibility.
- A potential overhead can arise with the simplified approach using rule engines, particularly when actors who process various issues. This is due to the need for each actor to share its belief snapshot with a RulesEngineManager, which then loads it into a context to execute the rule engine and extract the resource actor's subsequent intentions and actions. This overhead was identified through TokenRing benchmark, as described in [15], where experiments were conducted with 4,000 tokens, 4,000 nodes, and varying numbers of rounds.

Dynamic Extension

A user interface (UE in Table I) is required to interact with both the simulation and real entities to advance in maturity, the classification of Krizinger et al. [3]. The simulation is designed to operate in different contexts. It can process new issues directly from the live JIRA system, utilize preconfigured files with precise arrival timings for various topics, or run with synthetic data to observe simulation behaviors in a scenario where a large number of issues are obtained at each frequency or time distribution. This underlying architecture is based on the ISO 23247-2:2021 reference architecture. All timing information is transformed and kept within a specific time zone. The incorporation of rules allows hot reload, providing the flexibility to dynamically add or remove resources and states, and update their availability during runtime. It is also essential to understand how the model can evolve, including the introduction of new hubs, after organizational restructuring or testing a new way of working.

As a further extension layer, a local Large Language Model (LLM) with an embeddings database can summarize simulation results to suggest changes to users regarding simulation rules and configurations through the user interface. Related research also explores the combination of deep learning with DTs [25].

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:01:58 UTC from IEEE Xplore. Restrictions apply.

RESULTS

A publicly available data source is used to study the SDLC [10]. JIRA is a widely used tool that excels as an issue tracker and project management tool [26]. Using data from this public dataset ensures its potential applicability to many organizations managing issues and value streams with JIRA. The JIRA collection is tested using the 16 collections available on the public JIRA dataset. This dataset identifies 76 different states, which are then represented in a Petri net (Fig. 4). These states are key to building the simulation and the Finite State Machine associated with issue handling.

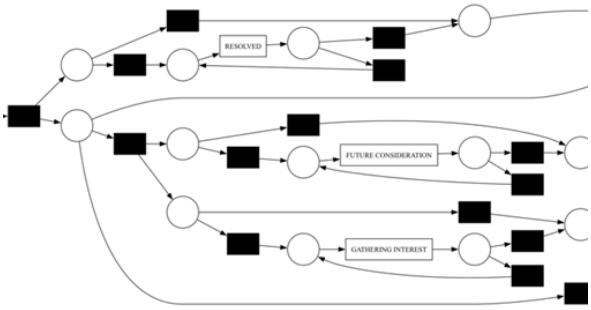

Figure 4. Zoomed in Petri Net region with transitions.

A preliminary analysis of the arrival process data reveals a mean inter-arrival time of approximately 39 minutes (2341.70 seconds), with a significantly lower median inter-arrival time of about 1.6 minutes (97.33 seconds). The substantial standard deviation observed suggests a non-Poisson behavior in issue arrivals. Based on these findings, the recommendation is to use an exponential distribution for simplicity, if acceptable, or a Weibull/Empirical distribution to better capture the inherent variability in the simulation.

A total of 3648 unique resources (developers, testers, etc.) with activity durations were identified. In this preliminary analysis, data from resource availability schedules are not considered. For each significant activity (e.g., "resolved", "closed", "open", "in progress"), median and mean sojourn times were calculated. We recommend modeling task durations per activity using statistical distributions (Weibull, Lognormal, or Empirical). For higher fidelity, these distributions could be differentiated by resource or role. For example, "resolved" activities show a median sojourn time of approximately 14,162 hours, with significant resource variance. Overall, the cycle time can be calculated with a mean case cycle time of approximately 1766 days (152 million seconds) and a median case cycle time of approximately 1511 days (130 million seconds).

These values provide a benchmark for validating simulation outputs against real historical performance. Considering the lengthy cycle time and the number of states involved, there is a high potential for improvement through DT simulation analysis of different issue-handling configurations. Consequently, the application of the DT in this context could be instrumental in identifying bottlenecks and optimizing rules for both resource management and state transitions.

The process described in Figure 2 has been tested in a production environment with 33 Scrum teams to predict which team should handle an incoming issue. The final calibrated LightGBM classifier achieved an overall accuracy of 93% on the test set. Furthermore, the macro-averaged precision, recall, and F1-score all reached 0.91, indicating strong and balanced performance across all classes. The weighted averages of these metrics were also 0.93, confirming that the classifier performs well even in the presence of class imbalance.

CONCLUSIONS AND FUTURE DIRECTIONS

This paper outlines the vision for the PRIMAD approach, a progressive methodology for constructing a DT for SDLC. Although this work is still in progress, preliminary results from separate experiments using Akka, pm4py, and LightGBM to validate the PRIMAD approach show promising outcomes for the integration of these tools. Recognizing DTs as a powerful outcome of digital transformation, we posit that this approach offers a highly suitable initial step for DT construction in the SDLC. The proliferation of AI agents on the SDLC may necessitate the adoption of DTs as a means to achieve a more autonomous value stream. Essentially, PRIMAD provides a practical starting point for organizations to begin experimenting with DTs in their software value streams.

Starting with real event data extraction and detailed statistical profiling ensures that the later simulation models are firmly grounded in empirical behavior rather than assumptions. Moreover, transforming event logs into actionable parameters (arrival rates, sojourn times, resource availability patterns) establishes the foundation for process-oriented simulations that are both realistic and calibratable over time. This method also supports progressive refinement: as the DT evolves, distributions can be dynamically adjusted based on updated data, and additional dynamic factors (such as blocking events, priorities, or multitasking effects) can be seamlessly introduced.

The agnostic nature of the PRIMAD approach to specific issue-tracking software, such as JIRA, makes it potentially applicable to other platforms, including Azure DevOps. This represents a natural evolutionary step for these systems, enabling the integration of intelligent, predictive, and adaptive capabilities. This evolution directly addresses current unmet needs in the industry, particularly as organizations increasingly embrace value stream management, predictive analysis for project delivery and maintenance, AI Copilots, and intelligent assistants. PRIMAD, with its modular, simulation-based, and explainable structure, could serve as the core engine driving these advanced functionalities.

The following steps of our research involve fully implementing and rigorously testing the PRIMAD approach across diverse projects within the publicly available JIRA dataset. The findings from this comprehensive implementation will be detailed in a forthcoming article. We believe that enabling a DT for SDLC is a crucial step towards facilitating the development of interconnected DTs capable of collaborative work and shared results. Therefore, the outcomes of the following article will be pivotal in establishing a definitive architecture for this transformative journey.

REFERENCES

[1] M. A. Guinea-Cabrera and J. A. Holgado-Terriza, “Digital Twins in Software Engineering - A Systematic Literature Review and Vision,” Applied Sciences, vol. 14, no. 3, p. 977, 2024.

[2] M. Grieves and J. Vickers, “Origins of the Digital Twin Concept,” Working Paper, 2016. [Online]. Available: https://doi.org/10.13140/RG.2.2.26367.61609.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:01:58 UTC from IEEE Xplore. Restrictions apply.

[3] W. Kritzinger, M. Kerner, G. Traar, J. Henjes, and W. Sihn, “Digital Twin in manufacturing: A categorical literature review and classification,” IFAC-PapersOnLine, vol. 51, pp. 1016-1022, 2018.

[4] T. West and M. R. Blackburn, “Is Digital Thread/Digital Twin Affordable? A Systemic Assessment of the Cost of DoD’s Latest Manhattan Project,” Procedia Comput. Sci., vol. 114, pp. 47-56, 2017.

[5] M. Caporuscio, F. Edrisi, M. Hallberg, A. Johannesson, C. Kopf, and D. Perez-Palacin, “Architectural Concerns for Digital Twin of the Organization,” in Software Architecture, ECSA 2020, Lecture Notes in Computer Science, vol. 12292, A. Jansen, I. Malavolta, H. Muccini, I. Ozkaya, and O. Zimmermann, Eds. Cham, Switzerland: Springer, 2020, pp. 267-282.

[6] Y. K. Dwivedi et al., “Artificial Intelligence (AI): Multidisciplinary Perspectives on Emerging Challenges, Opportunities, and Agenda for Research, Practice and Policy,” Int. J. Inf. Manage., vol. 57, p. 101994, 2021.

[7] R. Minerva, G. M. Lee, and N. Crespi, “Digital Twin in the IoT Context: A Survey on Technical Features, Scenarios, and Architectural Models,” Proc. IEEE, vol. 108, pp. 1785-1824, 2020.

[8] J. Epstein and R. Axtell, “Growing Artificial Societies. Social Science from the Bottom Up,” Washington, DC, USA: The Brookings Institution, 1996.

[9] M. Salgado, “Agent-Based Modelling,” 2013. [Online]. Available: https://doi.org/10.1007/978-94-6209-404-8_12.

[10] L. L. Montgomery, C. Llüders, and W. Maalej, “The Public JIRA Dataset,” 2022. [Online]. Available: https://zenodo.org/records/5901804.

[11] T. Brockhoff et al., “Process Prediction with Digital Twins,” in Proc. MODELS, 2021. [Online]. Available: https://doi.org/10.1109/MODELS-C53483.2021.00032.

[12] F. Leon, “ActressMAS, a .NET Multi-Agent Framework Inspired by the Actor Model,” Mathematics, vol. 10, no. 3, 2022. [Online]. Available: https://doi.org/10.3390/math10030382

[13] M. Wooldridge, An Introduction to Multi-Agent Systems, 2nd ed. Hoboken, NJ, USA: Wiley, 2009.

[14] H. Cazares Alegría and P. Pico-Valencia, “Software Agents Based on Machine Learning Technique: An Outlook from 2010 to 2023,” RCTA, vol. 1, no. 45, pp. 39-56, 2025.

[15] M. Mohajeri Parizi, G. Sileno, T. van Engers, and S. Klous, “Run, Agent, Run! Architecture and Benchmarking of Actor-Based Agents,” in Proc. 10th ACM SIGPLAN Int. Workshop on Programming Based on Actors, Agents, and Decentralized Control, New York, NY, USA, 2020, pp. 11-20.

[16] Akka Platform Documentation. [Online]. Available: https://doc.akka.io/

[17] F. Bellifemine, G. Caire, and D. Greenwood, Developing Multi-Agent Systems with JADE. Hoboken, NJ, USA: Wiley, 2007.

[18] N. Ford, R. Parsons, and P. Kua, Building Evolutionary Architectures: Support Constant Change. Sebastopol, CA, USA: O’Reilly Media, 2017.

[19] M. L. Abbott and M. T. Fisher, The Art of Scalability: Scalable Web Architecture, Processes, and Organizations for the Modern Enterprise, 2nd ed. Boston, MA, USA: Addison-Wesley, 2015.

[20] ThoughtWorks, Technology Radar, vol. 30, ThoughtWorks Publications, 2024.

[21] PM4PY Library. [Online]. Available: https://pypi.org/project/pm4py/

[22] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” in Proc. 31st Int. Conf. Neural Inf. Process. Syst. (NeurIPS), 2017, pp. 3149-3157.

[23] H. Han, W.-Y. Wang, and B.-H. Mao, “Borderline-SMOTE: A New Over-Sampling Method in Imbalanced Data Sets Learning,” in Proc. Int. Conf. Intelligent Comput. (ICIC), 2005, pp. 878- 887.

[24] OrientDB. Multimodel database. [Online]. Available: https://orientdb.dev/

[25] M. Kesti, “The Digital Twin of an Organization by Utilizing Reinforcing Deep Learning,” 2021. [Online]. Available: https://doi.org/10.5772/intechopen.96168.

[26] Atlassian, JIRA Software [Computer software]. [Online]. Available: https://www.atlassian.com/software/jira

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 22:01:58 UTC from IEEE Xplore. Restrictions apply.